

Es un simulador de lenguaje de programación (Pseudocódigo), sirve para aprender a programar por reglas estrictas

¡¡Ejemplo

Algoritmo (nombre)
Fin Algoritmo

(//) Comentario

1.000 de 1000

- N Entero = Número en decimal
- N Real = Número con decimales
- Carácter = Un letra / símbolo
- Cadena = Texto completo
- Lógico = Verdadero / Falso

Operadores

- + Suma
- Resta
- * Multiplicación

Aritméticos

- / División
- % Resto / módulo
- ^ Potencia

Operadores

Relacionales

- = Igual
- > Mayor que
- < Menor que
- >= Mayor o igual
- <= Menor o igual

- != Diferente de
- > Mayor que
- < Menor que
- >= Mayor o igual
- <= Menor o igual

Operadores

- Aritméticos
- + Suma
- Resta
- * Multiplicación

- / División
- % Resto
- ++ Incremento
- Decremento

- lógicos

- == Iguala
- != Diferente de
- > Mayor que

- < Menor que
- >= Mayor o igual
- <= Menor o igual

Entrada y salida de datos

ln = salto de línea

\\t = tabulación

\\n = Mostrar el símbolo \n

Bucles

do while $\rightarrow$ se ejecuta al menos una vez

while $\rightarrow$ se repite mientras se cumpla

for $\rightarrow$ se repite un número fijo de veces

Language C

- se organiza en funciones y bloques predefinidos
- se esconde casi a la vez de la medida de la memoria
- funciona en Windows, MacOS, Linux

int main() → es el punto de entrada del programa.
void

printf() → Muestra texto en pantalla

; → Tiene que ir después de terminar cada línea

Datos

int → Números enteros

double → Decimales más precisos

float → Números con decimales

char → Una letra / símbolo

Tuplas

- `Count (elementos)`: devuelve el número de veces que aparece un elemento en la tupla.
 - `Index (elementos)`: devuelve el índice de la primera aparición de un elemento en la tupla. Se puede especificar el inicio y fin de búsqueda.
 - `len (tupla)`: esta función devuelve la longitud de la tupla.
- ## Dictionarios
- `Keys`: devuelve una lista de todos los claves.
 - `Values`: devuelve la lista de todos los valores.
 - `Items`: devuelve la lista de todos los clave-valor.
 - `Update`: actualiza el diccionario con los pares clave-valor.

Conjuntos

- `add (elementos)`: agrega un elemento.
- `remove (elementos)`: elimina un elemento.
- `discard (elementos)`: elimina un elemento.
- `clear`: Elimina todos los elementos del conjunto.

Manejo de errores y excepciones

- `Error de sintaxis` → Cuando el código no sigue las reglas, como dividir puntos y comas.
- `Error de nombre` → ocurre cuando se hace referencia a un variable o función que no ha sido definido.
- `Error de tipo` → Cuando se realiza una operación con tipos de datos incompatibles, como número + cadena.
- `Error de índice` → Cuando se intenta acceder a un índice fuera de rango válido de una lista o cadena.

Bucles / loops

- **FOR**: se usa para iterar sobre una secuencia (como una lista, una tupla o una cadena) o cualquier objeto iterable
- **WHILE**: se utiliza para repetir un bloque de código mientras una condición sea verdadera
- Si una condición nunca se vuelve falsa, el bucle se ejecutará indefinidamente
- **Break**: se utiliza para salir prematuramente de un bucle, independientemente de la condición
- **Continue**: la instrucción continúa para saltar el resto del código dentro de un bucle
- **Pasa**: Operación nula que no hace nada. Se usa como marcador de posición cuando se requiere una instrucción sintácticamente.

Estructuras de datos

- **Listas**: Estructuras de datos mutables y ordenadas que permiten almacenar una colección de elementos

- **append (elemento)**: agrega un elemento al final.
- **insert (índice, elemento)**: inserta un elemento en una posición específica
- **remove (elemento)**: elimina la primera aparición de un elemento.
- **pop (índice)**: elimina y devuelve el elemento en una posición específica de la lista
- **sort**: ordena los elementos de la lista
- **Reverse**: invierte el orden de los elementos

Operadores

- Aritméticos
 - Suma (+)
 - Resta (-)
 - Multiplicación (*)
- División (/)
- Módulo (%)
- Exponenciación (**)

De comparación

- Igual a ($=$): Devuelve True si los valores son iguales.
- Diferente ($\neq$): Devuelve True si los valores son diferentes.
- Mayor que ($>$): Devuelve True si el primer valor es mayor.
- Menor que ($<$): Devuelve True si el primer valor es menor.
- Mayor o igual que ($\geq$): Devuelve True si el primer valor es mayor o igual.
- Menor o igual que ($\leq$): Devuelve True si el primer valor es menor o igual.

- Lógicos

- AND: True si ambas condiciones son verdaderas.
- OR: True si al menos una condición es verdadera.
- NOT: Invierte el valor de condición.

Estructuras de control

- Controlan el flujo

- IF: se utiliza para ejecutar un bloque de código si una condición es verdadera.
- IF - ELSE: Nos permite especificar un bloque de código alternativo que se ejecutará si la condición es falsa.
- IF - ELIF - ELSE: Nos permite especificar múltiples condiciones y bloques de código alternativos.

Python

Es un lenguaje de programación fácil de aprender leer.

Tipos de datos

- Enteros: no tienen decimal
- Flotantes: Puntos decimales
- Cadenas de texto: secuencias de caracteres entre comillas, se usan para representar texto
- Booleans: Representa los valores de verdad: True () y False (falso). se utilizan en condiciones y operadores lógicos

Variables

- Al nombrar las variables
- No contiene letras
 - No se usan palabras reservadas
 - Se recomienda usar nombres descriptivos

Operadores lógicos

Y los dos condicionales deben cumplirse
O Almeno uno de cumplir
NO Infierte el resultado

Fis mientras $\Rightarrow$ bucle y termina cuando
que la condicón se cumple